

Microsoft Certified: Security Operations Analyst Associate

Course

Defend against cyberthreats with Microsoft's security operations platform

---

Module 2

Mitigate threats using Microsoft Security Copilot

Modules in this learning path

COURSE 1

Introduction to generative AI and agents

Introduction

Generative AI, and technologies that implement it are increasingly in the public consciousness – even among people who don't work in technology roles or have a background in computer science or machine learning. The futurist and novelist Arthur C. Clarke is quoted as observing that "any sufficiently advanced technology is indistinguishable from magic". In the case of generative AI, it does seem to have an almost miraculous ability to produce human-like original content, including poetry, prose, and even computer code.

However, there's no wizardry involved in generative AI – just the application of mathematical techniques incrementally discovered and refined over many years of research into statistics, data science, and machine learning. You can gain a high-level understanding of how the magic trick is done by learning the core concepts and principles explored in this module. As you learn more about the generative AI technologies we have today, and how it powers a new generation of AI agents; you can help society imagine new possibilities for AI tomorrow.

Large language models (LLMs)

At the core of generative AI, large language models (LLMs) - and their more compact relations, small language models (SLMs) - encapsulate the linguistic and semantic relationships between the words and phrases in a vocabulary. The model can use these relationships to reason over natural language input and generate meaningful and relevant responses.

Fundamentally, LLMs are trained to generate completions based on prompts. Think of them as being super-powerful examples of the predictive text feature on many cellphones. A prompt starts a sequence of text predictions that results in a semantically correct completion. The trick is that the model understands the relationships between words and it can identify which words in the sequence so far are most likely to influence the next one; and use that to predict the most probable continuation of the sequence.

For example, consider the following sentence:

I heard a dog bark loudly at a cat

Now, suppose you only heard the first few words: "I heard a dog ...". You know that some of these words are more helpful clues as to what the next word might be than others. You know that "heard" and "dog" are strong indicators of what comes next, and that helps you narrow down the probabilities. You know that there's a good chance the sentence will continue as "I heard a dog bark".

You're able to guess the next word because:

You have a large vocabulary of words to draw from.

You've learned common linguistic structures, so you know how words relate to one another in meaningful sentences.
You have an understanding of semantic concepts associated with words - you know that something you heard must be a sound of some kind, and you know that there are specific sounds that are made by a dog.
So how do we train a model to have these same abilities?

Tokenization

The first step is to provide the model with a large vocabulary of words and phrases; and we do mean large. The latest generation of LLMs have vocabularies that consist of hundreds of thousands of tokens, based on large volumes of training data from across the Internet and other sources.

Wait a minute. Tokens?

While we tend to think of language in terms of words, LLMs break down their vocabulary into tokens. Tokens include words, but also sub-words (like the "un" in "unbelievable" and "unlikely"), punctuation, and other commonly used sequences of characters. The first step in training a large language model therefore is to break down the training text into its distinct tokens, and assign a unique integer identifier to each one, like this:

I (1)
heard (2)
a (3)
dog (4)
bark (5)
loudly (6)
at (7)
a (3) already assigned
cat (8)
and so on.

As you add more training data, more tokens will be added to the vocabulary and assigned identifiers; so you might end up with tokens for words like puppy, skateboard, car, and others.

Note

In this simple example, we've tokenized the example text based on words. In reality there would also be sub-words, punctuation, and other tokens.

Transforming tokens with a transformer

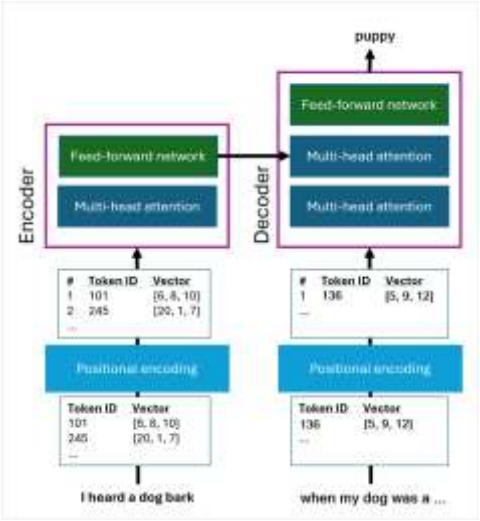
Now that we have a set of tokens with unique IDs, we need to find a way to relate them to one another. To do this, we assign each token a vector (an array of multiple numeric values, like [1, 23, 45]). Each vector has multiple numeric elements or dimensions, and we can use these to encode linguistic and semantic attributes of the token to help provide a great deal of information about what the token means and how it relates to other tokens, in an efficient format.

We need to transform the initial vector representations of the tokens into new vectors with linguistic and semantic characteristics embedded in them, based on the contexts in which they appear in the training data. Because the new vectors have semantic values embedded in them, we call them embeddings.

To accomplish this task, we use a transformer model. This kind of model consists of two "blocks":

An encoder block that creates the embeddings by applying a technique called attention. The attention layer examines each token in turn, and determines how it's influenced by the tokens around it. To make the encoding process more efficient, multi-head attention is used to evaluate multiple elements of the token in parallel and assign weights that can be used to calculate the new vector element values. The results of the attention layer are fed into a fully connected neural network to find the best vector representation of the embedding.

A decoder layer that uses the embeddings calculated by the encoder to determine the next most probable token in a sequence started by a prompt. The decoder also uses attention and a feed-forward neural network to make its predictions.



Note

We've greatly simplified the transformer architecture and process in the description and diagram. Don't worry too much about the specific details of how attention works - the key point is that it helps capture linguistic and semantic characteristics of each token based on the contexts in which it's used. If you want a deeper dive into the transformer architecture and how it uses attention, you can read the original Attention is all you need paper.

Initial vectors and positional encoding

Initially, the token vector values are assigned randomly, before being fed through the transformer to create embedding vectors. The token vectors are fed into the transformer along with a positional encoding that indicates where the token appears in the sequence of training text (we need to do this because the order in which tokens appear in the sequence is relevant to how they relate to one another). For example, our tokens might start off looking like this:

Token	Token ID	Position	Vector
I	1	1	[3, 7, 10]
heard	2	2	[2, 15, 1]
a	3	3	[9, 11, 1]
dog	4	4	[2, 7, 11]
bark	5	5	[9, 12, 0]
loudly	6	6	[3, 8, 13]
at	7	7	[5, 7, 10]
a	3	8	[9, 11, 1]
cat	8	9	[8, -6, 9]
...	...	...	...
puppy	127	45	[7, 7, -2]
car	128	56	[5, -5, 1]
skateboard	129	67	[4, 7, 14]

Note

We've kept things simple by using vectors with only three elements (which will help us visualize them in three-dimensions later). In reality, the vectors have thousands of elements.

Attention and embeddings

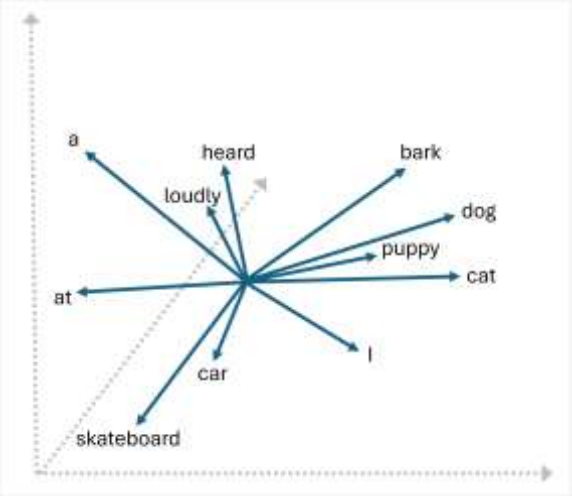
To determine the vector representations of tokens that include embedded contextual information, the transformer uses attention layers. An attention layer considers each token in turn, within the context of the sequence of tokens in which it appears. The tokens around the current one are weighted to reflect their influence and the weights are used to calculate the element values for the current token's embedding vector. For example, when considering the token "bark" in the context of "I heard a dog bark", the tokens for "heard" and "dog" will be assigned more weight than "I" or "a", since they're stronger indicators for "bark".

Initially, the model doesn't "know" which tokens influence others; but as it's exposed to larger volumes of text, it can iteratively learn which tokens commonly appear together, and start to find patterns that help assign values to the vector elements that reflect the linguistic and semantic characteristics of the tokens, based on their proximity and frequency of use together. The process is made more efficient by using multi-head attention to consider different elements of the vectors in parallel.

The result of the encoding process is a set of embeddings; vectors that include contextual information about how the tokens in the vocabulary relate to one another. A real transformer produces embeddings that include thousands of elements, but to keep things simple, let's stick to vectors with only three vectors in our example. The result of the encoding process for our vocabulary might look something like this:

Token	Token ID	Embedding
I	1	[2, 0, -1]
heard	2	[-2, 2, 4]
a	3	[-3, 5, 5]
dog	4	[10, 3, 2]
bark	5	[9, 2, 10]
loudly	6	[-3, 8, 3]
at	7	[-5, -1, 1]
cat	8	[10, 3, 1]
puppy	127	[5, 3, 2]
car	128	[-2, -2, 1]
skateboard	129	[-3, -2, 2]

We can think of the elements of the embeddings as dimensions in a multi-dimensional vector-space. In our simple example, our embeddings only have three elements, so we can visualize them as vectors in three-dimensional space, like this:



Because the dimensions are calculated based on how the tokens relate linguistically to one another, tokens that are used in similar contexts (and therefore have similar meanings) result in vectors with similar directions. For example, the embeddings for "dog" and "puppy" point in more or less the same direction, which isn't too different from the embedding for "cat"; but very different from the embedding for "skateboard" or "car". We can measure how close tokens are to one another semantically by calculating the cosine similarity of their vectors.

Predicting completions from prompts

Now that we have a set of embeddings that encapsulate the contextual relationship between tokens, we can use the decoder block of a transformer to iteratively predict the next word in a sequence based on a starting prompt.

Once again, attention is used to consider each token in context; but this time the context to be considered can only include the tokens that precede the token we're trying to predict. The decoder model is trained, using data for which we already have the full sequence, by applying a technique called masked attention; in which the tokens after the current token are ignored. Since we already know the next token during training, the transformer can compare it to the predicted token and adjust the learned weights in later training iterations to reduce the error in the model.

When predicting a new completion, for which the next tokens are unknown, the attention layers calculate possible vectors for the next token and the feed-forward network is used to help determine the most probable candidate. The predicted value is then added to the sequence, and the whole process repeats to predict the next token; and so on, until the decoder predicts that the sequence has ended.

For example, given the sequence "When my dog was a ...", the model will evaluate the tokens in the sequence so far, use attention to assign weights, and predict that the next most probable token is "puppy" rather than, say, "cat" or "skateboard".

Prompts

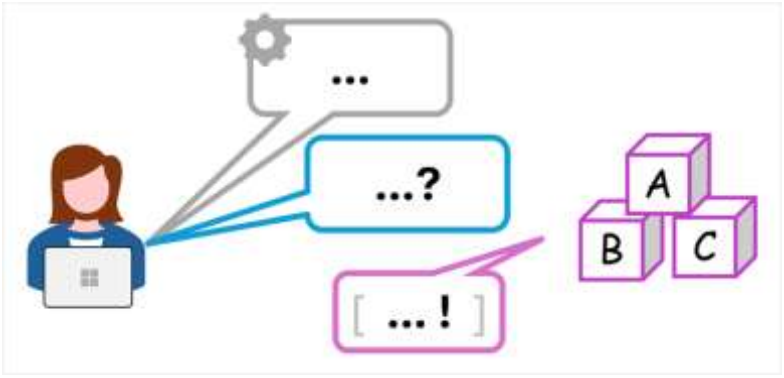
A prompt is simply the input you give to an LLM to get a response. It might be a question or a command, or just a casual comment to start a conversation. The model responds to a prompt with a completion.

Types of prompt

There are two main types of prompts:

System prompts that set the behavior and tone of the model, and any constraints it should adhere to. For example, "You're a helpful assistant that responds in a cheerful, friendly manner.". System prompts determine constraints and styles for the model's responses.

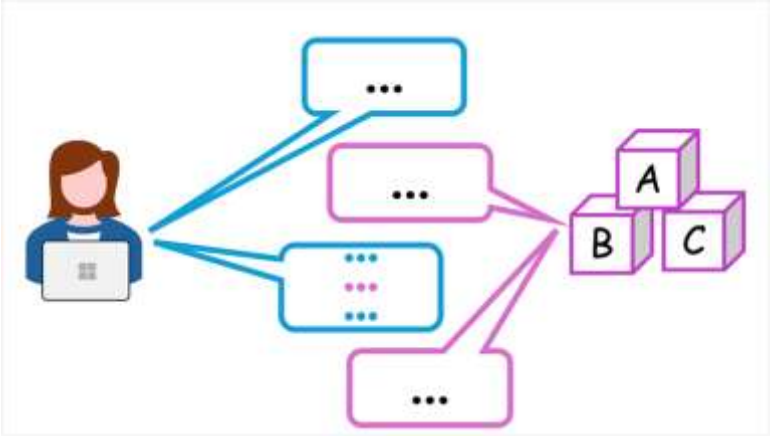
User prompts that elicit a response to a specific question or instruction. For example, "Summarize the key considerations for adopting generative AI described in GenAI_Considerations.docx for a corporate executive. Format the summary as no more than six bullet points with a professional tone.".



Usually, the system prompt is set by the application that uses the model. User prompts can be entered by a human user in a chat application; or in some cases generated by the application on the user's behalf. The model responds to user prompts while obeying the overall guidance in the system prompt.

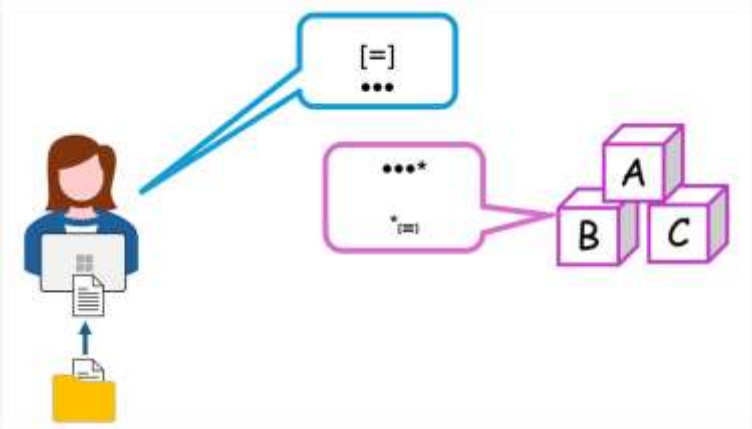
Conversation history

To keep a conversation consistent and relevant, generative AI apps often keep track of the conversation history; and include summarized versions of it in subsequent prompts. This ensures there's an ongoing context for the conversation that the model can build on.



For example, suppose the model responds to the system and user prompts described previously with the following completion:

Key considerations for adopting Generative AI include:
Establish Clear Governance and Responsible AI Policies
Prioritize High-Value Use Cases Across Corporate Functions
Mitigate Risks Around Privacy, Security, and Compliance
Foster Organizational Readiness and Cultural Adaptation
Measure Impact and Build Accountability
Stay Ahead of Strategic and Competitive Shifts
You could respond with a follow-up question, such as "What are common privacy-related risks?. The prompt would include the new question, but also the previous prompts and responses; providing context to the model so that it understands the question in relation to Generative AI adoption.
Retrieval augmented generation (RAG)
To add even more context, generative AI applications can use a technique called retrieval augmented generation (RAG). This approach involves retrieving information, like documents or emails, and using it to augment the prompt with relevant data. The response generated by the model is then grounded in the information that was provided.
For example, suppose you submit a prompt like "What's the maximum I can claim for travel expenses on a business trip?". With no other information, a model will respond with a generic answer - probably telling you to consult your organization's expenses policy documentation. A better solution would be to build an expenses assistant app that initially queries the organization's expenses policy documentation, retrieving sections related to "travel expenses"; and then includes the retrieved information in the prompt that is sent to the model, along with your original question. Now the model can use the expenses policy information in the prompt to provide context, and respond with a more relevant answer.



Tips for better prompts
The quality of responses from generative AI assistants not only depends on the language model used, but on the prompts you submit to it.

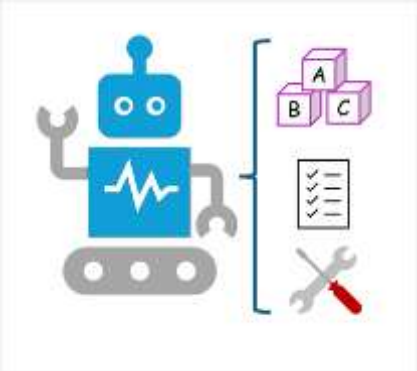


To get better results from your prompts:
Be clear and specific – prompts with explicit instructions or questions work better than vague language.
Add context - mention the topic, audience, or format you want.
Use examples, If you want a certain style, provide an example of what you mean.
Ask for structure, Like bullet points, tables, or numbered lists.
Using well-designed prompts can make a huge difference to the results you'll get from your generative AI model.
AI agents

Imagine having a digital assistant that doesn't just answer questions, but actually gets things done! Welcome to the world of AI agents.

Agents are software applications built on generative AI that can reason over and generate natural language, automate tasks by using tools, and respond to contextual conditions to take appropriate action.

Components of an AI agent



AI agents have three key elements:

A large language model: This is the agent's brain; using generative AI for language understanding and reasoning.

Instructions: A system prompt that defines the agent's role and behavior. Think of it as the agent's job description.

Tools: These are what the agent uses to interact with the world. Tools can include:

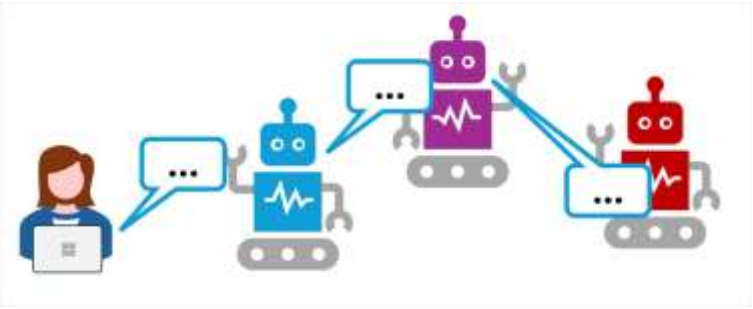
Knowledge tools that provide access to information, like search engines or databases.

Action tools that enable the agent to perform tasks, such as sending emails, updating calendars, or controlling devices.

With these capabilities, AI agents can take on the role of digital assistants that intelligently automate tasks and collaborate with you to work smarter and more efficiently.

Multi-agent systems

Agents can also work with one another, in multi-agent systems. Instead of one agent doing everything, multiple agents can collaborate—each with its own specialty. One might gather data, another might analyze it, and a third might take action. Together, they form an AI-powered workforce that can handle complex workflows, just like a human team.



Agents communicate with each other through prompts, using generative AI to determine what tasks are required and which agents are responsible for completing them.

Agentic AI is set to be the next advance in how we use technology to find information and get work done.

Explore generative AI

In this exercise, you'll use a chat playground to interact with a generative AI model, and explore how it could be used to support an employee expenses agent that helps employees with expense claim policies and processes.

To complete this exercise, you need a modern browser on a computer with sufficient hardware resources to load and run the models used by the Chat Playground and Model Coder apps. On older or low-spec computers, the apps may run very slowly or experience errors.

Minimum spec

64-bit CPU, 8 cores

GPU (recommended)

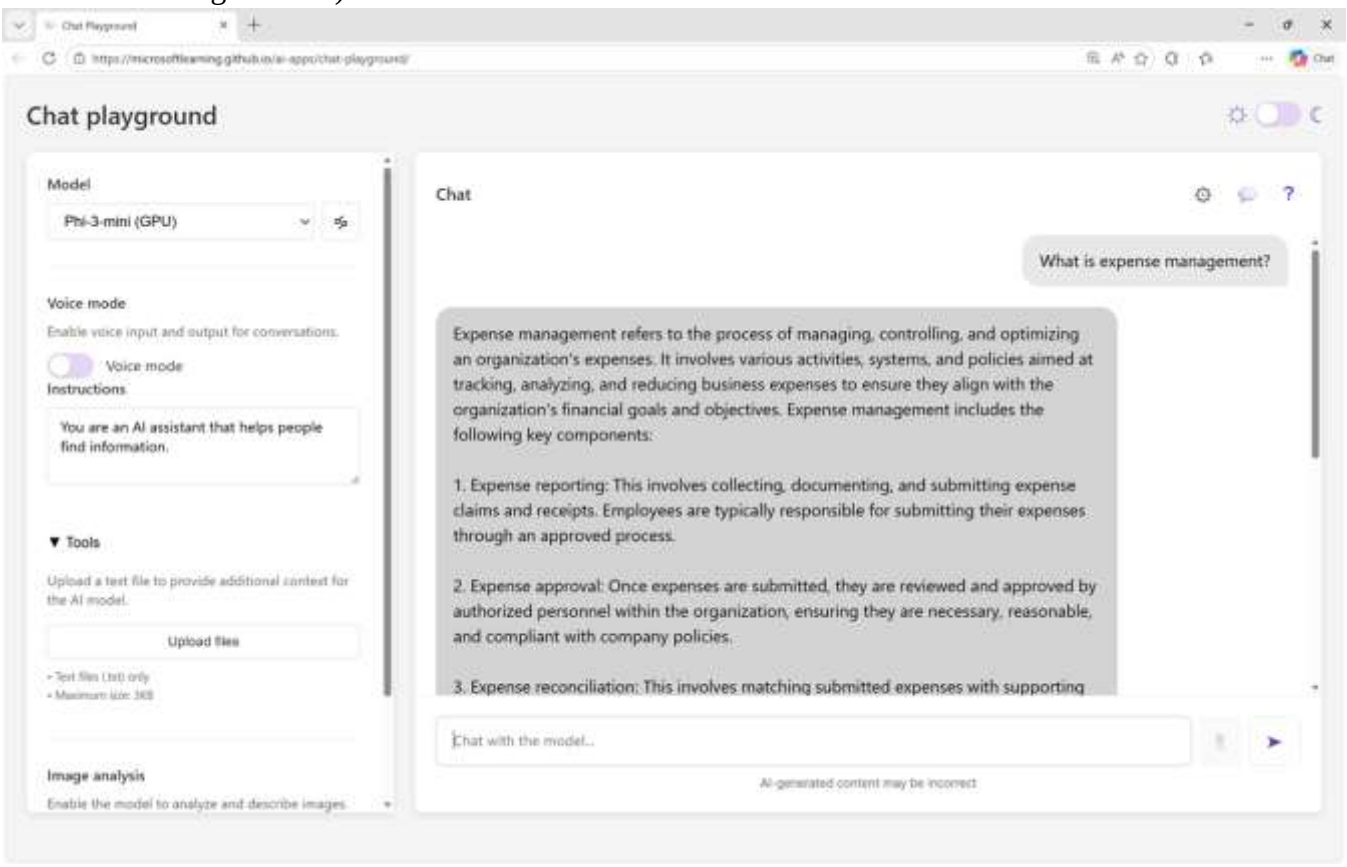
8+ GB system RAM (16 GB recommended)

Enough storage to cache ~300MB–800MB model assets

Latest Chrome / Edge / Firefox with WASM SIMD enabled/available (WebGPU support is recommended; a WASM-based fallback is provided)

If your computer does not meet these requirements, the AI model may not run successfully. However, the apps support a failsafe mode in which no model is used; which you may be able to use.
This exercise should take approximately 15 minutes to complete.
Chat with a model
Let's start by using a chat interface to submit prompts to a generative AI model. In this exercise, we'll use a small language model that is useful for general chat solutions in low bandwidth scenarios.
Note: If your browser supports WebGPU, the chat playground uses the Microsoft Phi 3.5 Mini model running on your computer's GPU. If not, the model runs on CPU - with reduced response-generation quality. If that fails, a basic mode with no model and responses retrieved from Wikipedia is activated. Performance may vary depending on the available memory in your computer and your network bandwidth to download the model. After opening the app, use the ? (About this app) icon in the chat area to find out more.
In a web browser, open the Chat Playground at https://aka.ms/chat-playground.
Wait for the model to download and initialize.
Tip: The first time you download a model, it may take a few minutes. Subsequent downloads will be faster.

If the model is taking a long time to load, you can cancel and start in basic mode. You can switch between available models at any time in the Model list.
When the model is ready, enter the prompt What is expense management?, and review the response (which may take some time to be generated).



Tip: You can switch between light and dark themes using the ☼ / ☾ toggle at the top right.
In the pane on the left, in the Instructions text area, change the model's instructions to You are an AI assistant that provides succinct answers to business expense-related questions.
Instructions, sometimes known as a system prompt, are used to provide the model with an overall context for its responses. You can use the system prompt to provide guidelines about format, style, and constraints about what the model should and should not include in its responses.
At the top of the chat pane, use the New chat (🗨) button to restart the conversation. This removes all conversation history.
Now try the same prompt as before (What is expense management?) and review the output.
Enter another prompt, such as Tell me about per-diem allowances. and review the response.
Now try a follow-up question: How are they reimbursed?

Note: Generative AI chat applications often include chat history in the prompt; so the context of the conversation is retained between messages (for example, in the follow-up prompt How are they reimbursed?, “they” is interpreted as relating to per-diem allowances).

In Basic mode, the conversation history is not retained; so the follow up prompt results in a new Wikipedia query.

Extend the model’s capabilities with tools

Generative AI is the foundation for agentic solutions; in which AI agents can assist you and act on your behalf. To accomplish this, agents need tools that allow them to access specific knowledge and perform tasks.

In the case of our expenses assistant, we can equip the agent with tools that enable it to access relevant information beyond the data with which the model was trained.

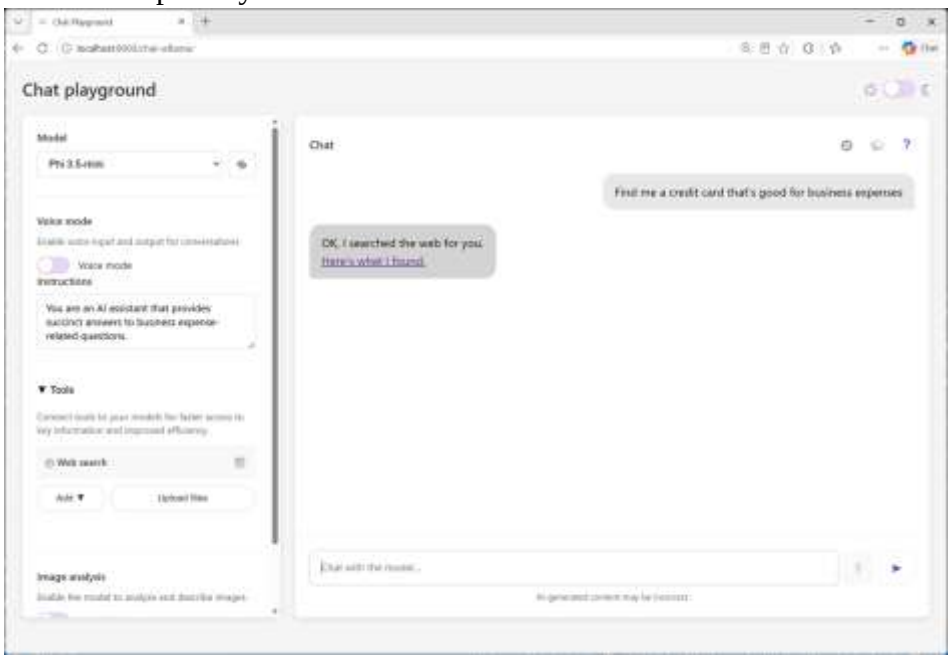
In the chat playground, at the top of the chat pane, use the New chat (🗨) button to restart the conversation.

In the pane on the left, expand the Tools section if it’s not already expanded.

In the Add menu, select Web search to add a web search tool.

Enter the prompt Find me a credit card that's good for business expenses.

The agent uses its web search capability to search the web and find relevant information.



The web search tool is great for finding general information on the public Internet; but we need our agent to be able to get information from corporate expense policy documentation.

Open a new browser tab, and view the expenses guide at https://aka.ms/expenses-txt. We’ll use this to ground the model, so it has some context for questions about expenses.

Note: This is a very small document for the purposes of this exercise. In a real scenario, an AI agent might have access to large volumes of data; usually in the form of a vector index.

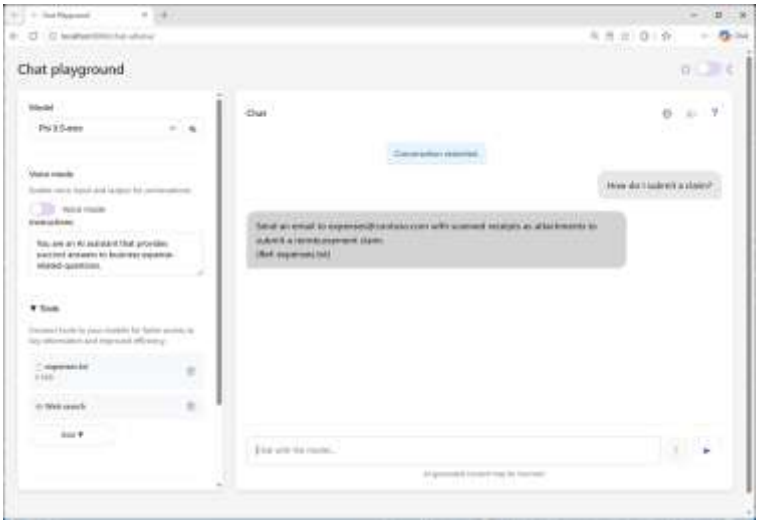
Save the expenses.txt file on your local computer.

Return to the tab containing the chat playground, and in the pane on the left, in the Tools section, add the File search tool (or select Upload files).

Upload the expenses.txt file. The chat is automatically restarted.

Enter the prompt How do I submit a claim? and view the response.

This time the response should be informed by the information in the expenses data source.



Try a few more expenses-related prompts - for example, How much can I spend on a taxi? or What about a hotel?.
Note: The small amount of data and the limited capabilities of the small models used in this exercise may result in some inaccurate responses; but the principle of retrieving contextual information, using it to augment the prompt, and generating responses based on the data is a common pattern in generative AI solutions known as Retrieval Augmented Generation (or RAG).

Explore client code

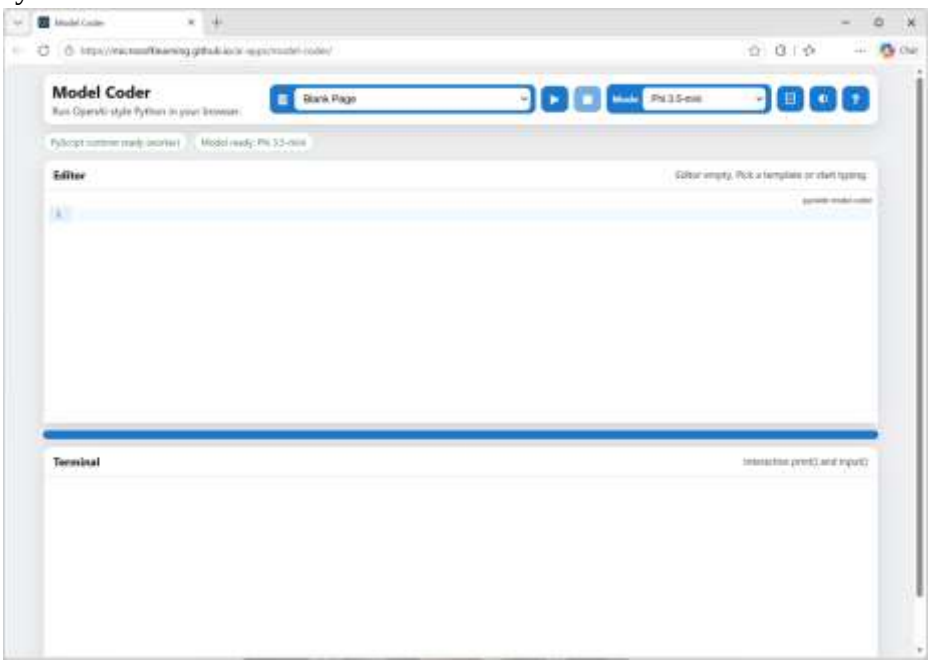
You've seen how models and agents can be used in a pre-provided chat playground, but how do developers build apps and agents that submit prompts to models and process responses?

One of the most commonly used application programming interfaces (APIs) used to develop apps that work with LLMs is the OpenAI API - and in particular the Python SDK for the OpenAI API.

Navigate away from the Chat Playground app to the Model Coder app at https://aka.ms/model-coder and wait for the Python environment and model to be ready.

Note: As with the chat playground, the first time the model is loaded it may take a minute or so.

If the model is taking a long time to load, you can cancel and start in basic mode. You can switch between available models at any time in the Mode list.



Tip: You can switch between light and dark themes using the ◑ icon at the top right.

This app provides an in-browser sandbox with a Python library that encapsulates the most common classes in the OpenAI SDK. You'll use it to write and run real Python code that submits prompts to a local LLM running in the browser.

When the model has loaded, change the Blank Page sample to the Simple chat (Responses API) sample; and review the code in the Editor pane. It should be similar to this:

code

```
# import namespace
```

```
from openai import OpenAI

def main():

    try:
        # Configuration settings
        endpoint = "https://local/openai"
        key = "key123"
        model_name = "phi"

        # Initialize the OpenAI client
        openai_client = OpenAI(
            base_url=endpoint,
            api_key=key
        )

        # Loop until the user wants to quit
        while True:
            input_text = input('\nEnter a prompt (or type "quit" to exit): ')
            if input_text.lower() == "quit":
                print("Goodbye!")
                break
            if len(input_text) == 0:
                print("Please enter a prompt.")
                continue

            # Get a response
            response = openai_client.responses.create(
                    model=model_name,
                    instructions="You are a helpful AI assistant that provides concise responses.",
                    input=input_text
            )
            print(response.output_text)


    except Exception as ex:
        print(ex)

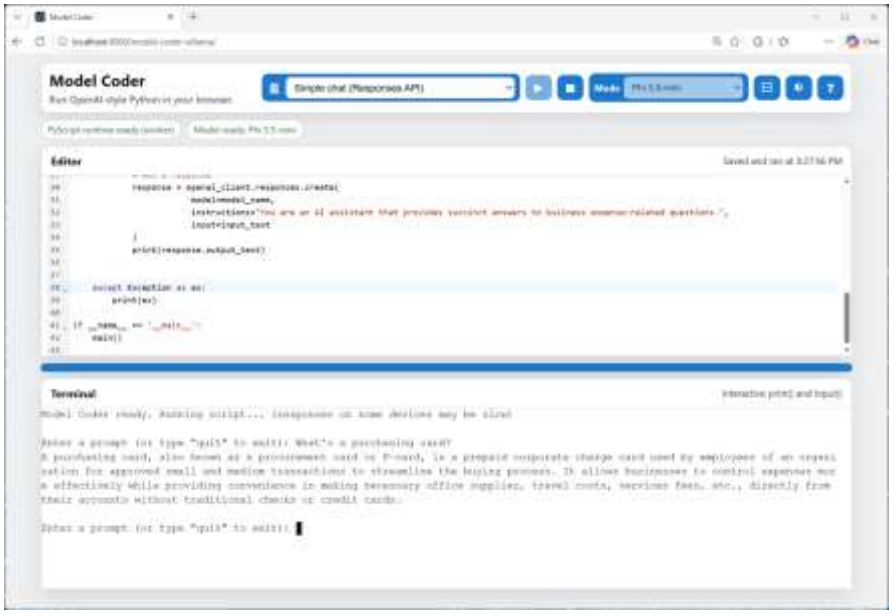
if __name__ == '__main__':
    main()
```

This code uses the OpenAI Responses API, which is commonly used to submit prompts to models and agents. In the code editor, change the Instructions parameter used to create a response from the model to You are an AI assistant that provides succinct answers to business expense-related questions. (the same system prompt we used in Chat Playground). The relevant section of the code should now look like this:

code

```
# Get a response
response = openai_client.responses.create(
        model=model_name,
        instructions="You are an AI assistant that provides succinct answers to business expense-related questions.",
        input=input_text
```

)
print(response.output_text)
Use the ▶ (Run code) button on the toolbar to run the Python code.
The code runs in the Terminal pane at the bottom of the screen (it may take a minute or so to run).
When prompted, enter a question about expense categories; such as:
code
What's a purchasing card?
Wait for the response, and then review the answer that was returned.



Note: The model used in this app is a small language model with limited training data and a small context window. Responses may not be accurate. However, the point of the exercise is to explore the OpenAI SDK syntax to submit prompts and receive responses.
Continue the conversation, or enter quit to stop the Python code.
Summary
In this exercise, you explored a generative AI model in a chat playground. You've seen how a model's responses can be affected by changing the system prompt, configuring model parameters, and by adding data. Finally, you've explored how developers can build generative AI client applications through OpenAI-compatible APIs in Python.
The interface and techniques used in this exercise are similar to those in Microsoft Foundry portal; a platform for building AI apps and agents in the Microsoft Azure cloud. You can use the OpenAI SDK to connect to Microsoft Foundry endpoints and work with your models and agents there.